

NUMBER PUZZLE GAME IN C++



■ VEERANNA V.K. AND
JANAKIDEVI S.J.

Number puzzles like Sudoku are now a regular feature of many newspapers. And many of us have become addicted to solving them. These puzzles require you to apply the best permutation and combination logic.

For instance, in Sudoku, you are provided with a partially completed 9×9 grid. You need to fill this grid such that each column, each row and each of the nine 3×3 boxes contain the digits from '1' to '9' only one time each.

Here we present a number puzzle that is dynamic compared to Sudoku. The procedure of playing the game is very simple. At first, when the program is run, 15 square-shaped boxes numbered from '1' to '15' and a green-coloured square box with asterisk in it are displayed on the screen (refer Fig. 2). The numbers and asterisk are displayed at random, not in sequence. You have to move around the green-coloured asterisk to arrange the boxes '1' through '15' in sequence with the asterisk at the last position. An inter-



Fig. 1: Screenshot of main program menu



Fig. 2: Demo program output of puzzle game

esting demo is added in the program to make you understand the logic of the game and arrange the numbers as shown in Fig. 2.

Once you successfully arrange the numbers within 50 moves, you will get an Indian tri-colour flag as shown in Fig. 3. If you fail, you will get a distorted image of the flag along



Fig. 3: End of the game with the flag

with the message 'Game over, you lost.' So keep trying to get the fine Indian flag.

The program is written in C++ and tested in Windows XP. In order to use graphics facilities, all the FONT and BGI graphics drivers must be placed in the working directory of 'TC.EXE' rather than in BGI directory. The program code consists of many functions, but these functions are very simple and do not need much explanation as the names themselves imply the purpose of each function.

EFY note. The source code of this program is included in this month's EFY-CD.

PZGAME.CPP

```
#include "iostream.h"
#include <graphics.h>
#include <stdlib.h>
#include <stdio.h>
#include <conio.h>
#include <alloc.h>
#include <ctype.h>
#include <dos.h>
#include <stdlib.h>
#include <math.h>
char *num[16]= {"01", "02", "03", "04",
"05", "06", "07", "08", "09", "10",
"11", "12", "13", "14", "15", ""};
char *esnum[16]= {"01", "06", "02",
"04", "05", "", "03", "07", "10", "11",
"12", "08", "09", "13", "14", "15"};
char *hnum[16]= {"02", "07", "04", "08",
"10", "14", "12", "15", "01", "09",
"03", "", "05", "13", "06", "11"};
```

```
char *dnum[16]= {"01", "03", "04",
"08", "06", "02", "07", "", "05", "09",
"11", "12", "13", "10", "14", "15"};
char *snum[16];

int matchnb[16]= {0,1,2,3,4,5,6,7,8,9,
10,11,12,13,14,15};
int dloc[22]= {7,3,3,2,2,1,1,5,4,4,8,8,
9,9,13,13,1,4,14,15};
struct square {int left; int top; int right;
int bottom; char* value;
int valflag; char image;}
square s[15];
char vmessage[25], vbuff[1024], nkey;
void far *ptr;
int key, c = 3, c = 3, result;
```

```
void game_description();
void gameinst();
void show_screen();
void show_flag();
void init_boxes(square sq[]);
void draw_boxes(square sq[]);
void draw_flag(square sq[]);
void flag_without_fill(square sq[]);
void simple_play();
void hard_play();
void demo_play();
void vdemo();
void flag_india(square sq[]);
void shuffle_boxes(square sq[], char *snum);
void match_result(square sq[]);
void shuffled_flag(square sq[]);
void vplay();
int getkey();
```